Math Behind the Spotify Genre Classifier

A walkthrough of the mathematics used in the PyTorch MLP that predicts a track's genre from 14 audio features.

1. Data preparation

Label encoding: each genre name becomes an integer y_i in {0, ..., C-1}, where C is the number of genres.

Stratified split: 15% test, then 15% of the remainder for validation. Each split keeps the same class proportions as the full set.

Standardization (z-score). Mean and standard deviation are computed on the training set only, to avoid leakage:

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{ij}, \qquad \sigma_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_{ij}-\mu_j)^2}$$

$$\tilde{x}_{ij} = \frac{x_{ij}-\mu_j}{\sigma_j + 10^{-8}}$$

The same mean and standard deviation are applied to validation and test.

2. The network (MLP: 14 → 128 → 64 → C)

Each linear layer is an affine map:

$$\mathbf{z}^{(l)} = W^{(l)}\,\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$$

ReLU activation:

$$\mathrm{ReLU}(z) = \max(0,\, z)$$

Dropout (p = 0.2, training only). With a mask m_k drawn from Bernoulli(0.8):

$$h'_k = \frac{m_k\, h_k}{0.8}$$

At eval time, dropout is the identity. The 1/0.8 scaling keeps the expected activation the same in both modes.

Full forward pass:

$$\mathbf{h}^{(1)} = \mathrm{Drop}\big(\mathrm{ReLU}(W^{(1)}\tilde{\mathbf{x}} + \mathbf{b}^{(1)})\big), \quad W^{(1)} \in \mathbb{R}^{128\times 14}$$

$$\mathbf{h}^{(2)} = \mathrm{Drop}\big(\mathrm{ReLU}(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})\big), \quad W^{(2)} \in \mathbb{R}^{64\times 128}$$

$$\mathbf{o} = W^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}, \quad W^{(3)} \in \mathbb{R}^{C\times 64}$$

Here **o** holds the logits.

Parameter count:

$$(14 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64C + C) = 10176 + 65C$$

For C = 114 that gives about 17.6k parameters.

3. Loss: softmax + cross-entropy

Softmax turns logits into probabilities:

$$p_c = \frac{e^{o_c}}{\sum_{k=1}^{C} e^{o_k}}$$

Cross-entropy (nn.CrossEntropyLoss applies log-softmax internally):

$$\mathcal{L}_i = -\log p_{y_i} = -o_{y_i} + \log \sum_k e^{o_k}$$

$$\mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}_i, \quad B = 256$$

Reported epoch loss (weighted by batch size, so the smaller last batch counts correctly):

$$\text{loss}_{\text{epoch}} = \frac{1}{N} \sum_{\text{batches}} \mathcal{L}_{\text{batch}} \cdot |\text{batch}|$$

The gradient with respect to the logits is simple:

$$\frac{\partial \mathcal{L}_i}{\partial o_c} = p_c - 1[c = y_i]$$

4. Optimizer: Adam

With gradient g_t and PyTorch defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e\text{-}8$:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

5. Learning-rate schedule (MultiStepLR)

$$\eta(e) = \eta_0 \cdot \gamma^{\left|\{m \in \{12, 20\} : m \le e\}\right|}, \quad \eta_0 = 10^{-3},\ \gamma = 0.5$$

Epoch index e	Learning rate
0 - 11	1e-3
12 - 19	5e-4
20 - 29	2.5e-4

6. Evaluation metrics

Accuracy:

$$\mathrm{acc} = \frac{1}{N}\sum_{i=1}^{N} 1[\arg\max_c o_{i,c} = y_i]$$

Top-5 accuracy:

$$\mathrm{top5} = \frac{1}{N}\sum_{i=1}^{N} 1[y_i \in \mathrm{TopK}_5(\mathbf{o}_i)]$$

Chance level (random guessing, balanced classes) is 1/C. For C = 114, that is about 0.88%.

Confusion matrix: CM_{ab} is the number of samples with true class a predicted as class b:

$$CM_{ab} = \sum_i 1[y_i = a]\ 1[\hat{y}_i = b]$$

Per-genre accuracy (recall): the diagonal divided by the row sum:

$$\mathrm{acc}_a = \frac{CM_{aa}}{\sum_b CM_{ab}}$$

The plot shows the median of the per-genre accuracies in its title, and bars are sorted with argsort in ascending order of per-genre accuracy.